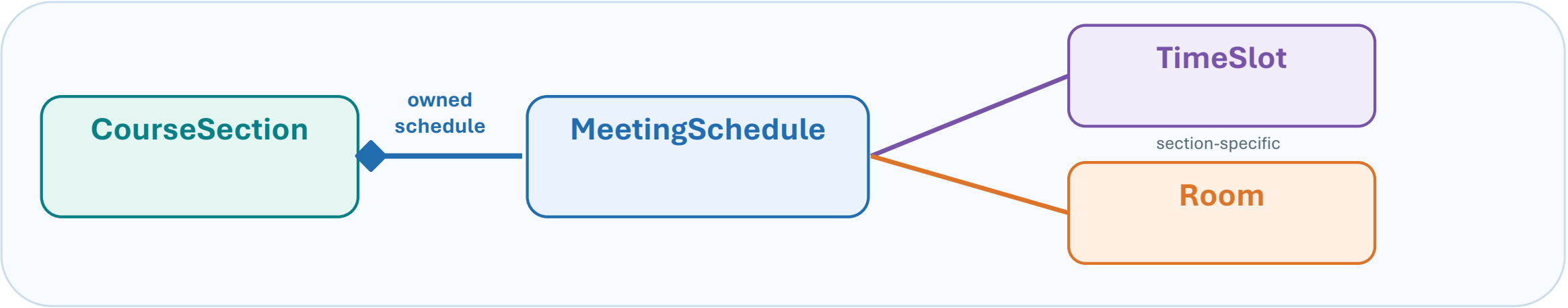
LECTURE 10

Multi-Level Composition

Ownership • Relationship Semantics • Cloning & Copying



One Java field can mean sharing, collaboration, or strong ownership—domain rules decide.

Dr. Muhammad Shahid Bhatti

Bridge from Lecture 9

CONTINUITY

Lecture 9 — we established

- Fields and arrays store object references.
- Objects can be passed to and returned from methods.
- Composition expresses a HAS-A relationship.
- Composition is often better than inheritance when the relationship is not truly IS-A.

Lecture 10 — we deepen the model

- Composition can be nested across several levels.
- Not all object relationships have the same ownership strength.
- Cohesion and coupling help us judge class quality.
- Object lifetime matters.
- Cloning reveals why nested references require careful copy semantics.

Today the question is no longer only "Does A have B?" — it is "Who owns B, how long does B live, and how tightly are these classes connected?"

LECTURE 10

Lecture 10 — topic map

COURSE PLAN

1 **Multi-level composition**

Nested HAS-A relationships / object graphs

2 **Relationship strength**

Association → aggregation → composition

3 **Ownership & lifetime**

Who creates, controls, and destroys the part?

4 **Design quality**

Cohesion + coupling as class design criteria

5 **Composition vs inheritance**

Prefer composition when no true IS-A exists

6 **Cloning model**

Object.clone() + Cloneable marker interface

LECTURE 10

OUTCOMES

Learning outcomes

MODEL

Draw and explain a nested object graph with more than one composition level.

CLASSIFY

Distinguish association, aggregation, and composition using ownership and lifetime reasoning.

DESIGN

Use cohesion and coupling to critique and refactor class designs.

DECIDE

Choose composition over inheritance when the domain does not express a true IS-A relation.

COPY

Explain Java cloning, Cloneable, shallow copy, and strategies for deeper copies.

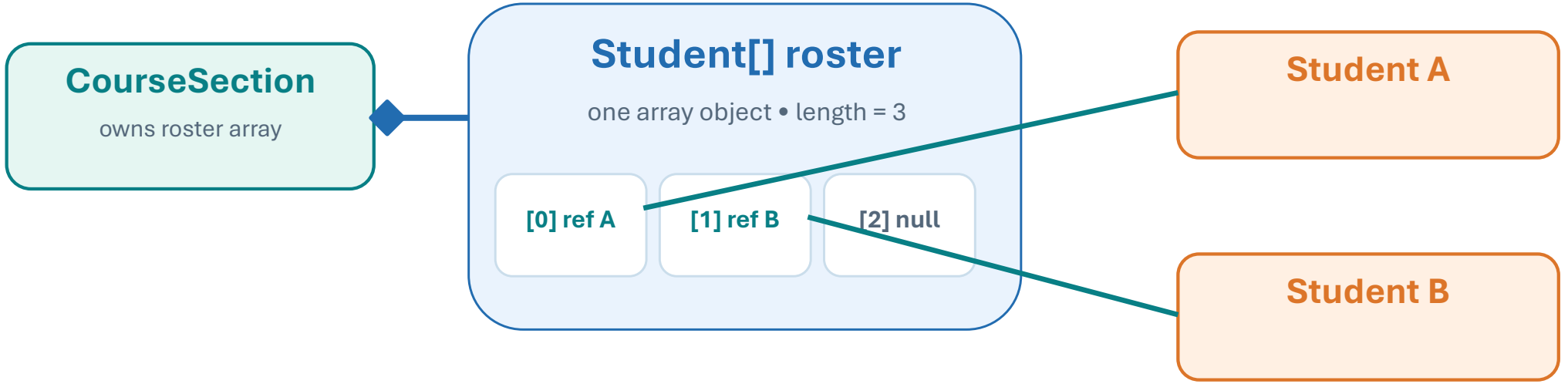
CODE

Build, compile, trace, and test a multi-object Java example.

Bridge from Lecture 9 — an array is not its elements

RECALL

A class can own an array object without owning the objects referenced by its elements.



No object is created for the null slot.

Array slots hold references; students retain independent lifetimes. Diamond denotes ownership of the array—not its elements.

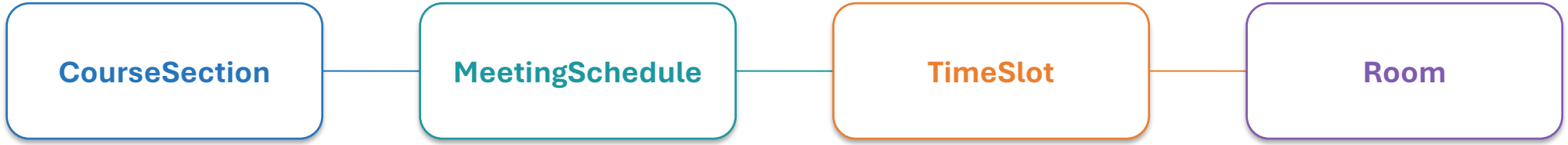
MENTAL MODEL

One reference creates a link; many references create an object graph

Simple composition



Nested composition / object graph



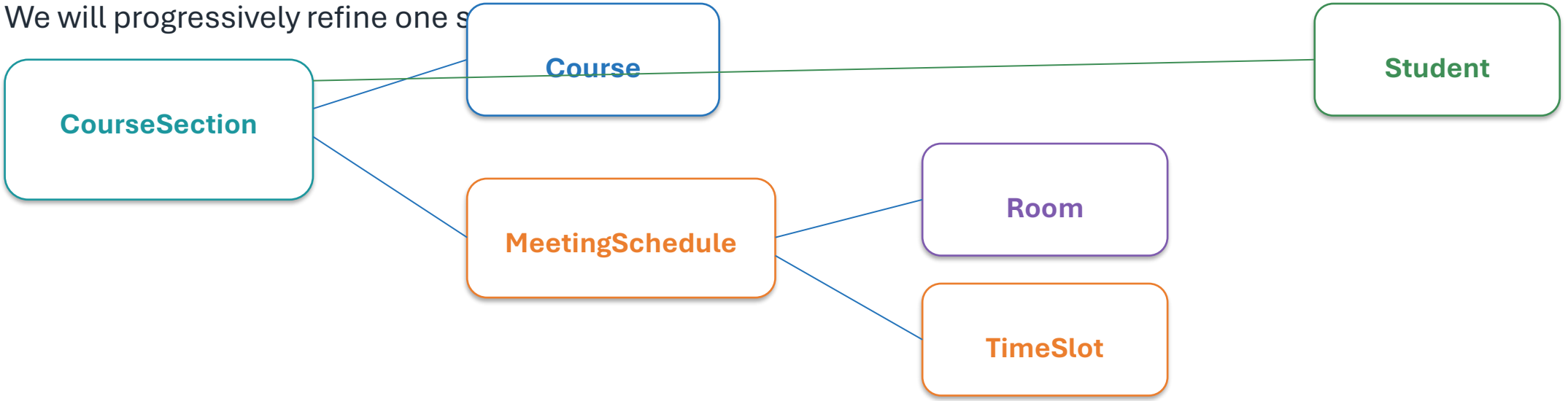
A useful design question at each link:

“Is this merely a reference, a shared part, or a strongly owned part whose lifetime depends on its whole?”

RUNNING MODEL

Running example — a CourseSection object graph

We will progressively refine one s



Semantics, not just arrows:

Course is referenced by the section but exists independently.

MeetingSchedule can be owned by the section.

A schedule can itself contain a Room and a TimeSlot.

Students participate in a section; the section does not “own” their lifetime.

Multi-level object structure: references across levels

NESTED COMPOSITION

Level 1

CourseSection has a MeetingSchedule.

Level 2

MeetingSchedule has a Room and TimeSlot.

Level 3

A Room may itself contain room-specific resources.

Nested composition creates a navigable object graph:

```
OBJECT GRAPH

CourseSection
  └── MeetingSchedule
        ├── Room
        └── TimeSlot
```

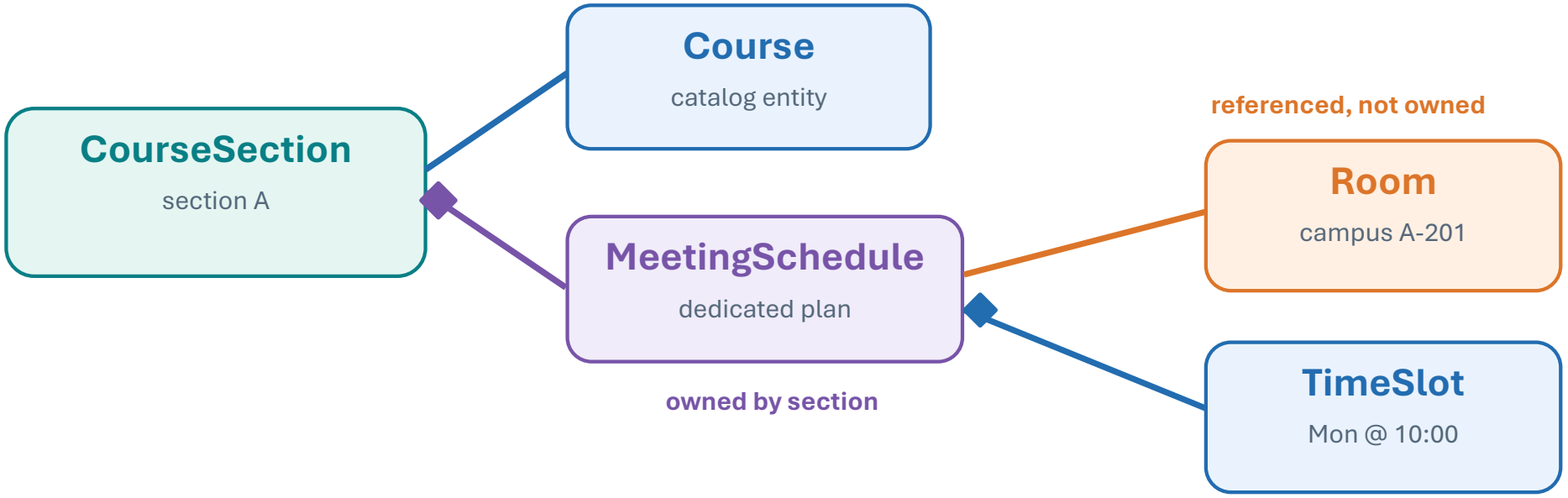
Important:

"Nested" describes the object references. It does not mean every relationship in the graph is automatically strong composition.

Ownership must still be justified link-by-link.

Object graph vs. ownership

An object reference is a link—not automatic ownership.



Modeling decision: a section-specific MeetingSchedule may be owned; Room remains an independent campus resource.

CODE

Code the innermost value object first

TimeSlot.java

```
public class TimeSlot {
    private final String day;
    private final String startTime;

    public TimeSlot(String day, String startTime) {
        this.day = day;
        this.startTime = startTime;
    }
    public TimeSlot(TimeSlot other) {
        this(other.day, other.startTime);
    }
    public String describe() {
        return day + " @ " + startTime;
    }
}
```

Why start here?

It has a small cohesive responsibility: represent and describe one scheduled time.

Design signal

Fields are final because the example treats the slot as fixed after construction.

Live question

Should TimeSlot know anything about CourseSection? Usually no — that would increase coupling unnecessarily.

Level 2 — MeetingSchedule references Room and TimeSlot

CODE

MeetingSchedule.java

```
public class MeetingSchedule {
    private final Room room;
    private final TimeSlot slot;

    public MeetingSchedule(Room room, TimeSlot slot) {
        this.room = room;
        this.slot = slot;
    }

    public Room getRoom() { return room; }
    public TimeSlot getSlot() { return slot; }

    public String describe() {
        return slot.describe() + " in " + room.getCode();
    }
}
```

Nested references

MeetingSchedule now points to two other objects.

Question

A real Room usually exists independently. A dedicated TimeSlot may be owned by this MeetingSchedule, depending on the domain.

Key idea

Java syntax alone does not tell us relationship strength; domain semantics do.

Level 1 — CourseSection composes MeetingSchedule

CODE

CourseSection.java

```
public class CourseSection {
    private final String sectionId;
    private final Course course;
    private final MeetingSchedule schedule;

    public CourseSection(String sectionId, Course course,
                         MeetingSchedule schedule) {
        this.sectionId = sectionId;
        this.course = course;
        this.schedule = schedule;
    }

    public String describe() {
        return sectionId + " | " + course.getCode()
             + " | " + schedule.describe();
    }
}
```

Trace the chain

section → schedule → slot / room

Ownership question

A section-specific MeetingSchedule is a good candidate for strong ownership because it exists to describe that section.

Independent collaborator

Course itself normally exists beyond a particular section, so treat it as an independent referenced object.

Live coding — build the nested object graph

LIVE CODING

Main.java

```
Course oop = new Course("CSC241", "Object-Oriented
Programming");
Room room = new Room("A-201");
TimeSlot slot = new TimeSlot("Monday", "10:00");

MeetingSchedule schedule = new MeetingSchedule(room,
slot);
CourseSection section = new CourseSection("A", oop,
schedule);

System.out.println(section.describe());
```

PREDICT

How many distinct objects are created?

TRACE

Which object directly references Room?

EXPLAIN

Why can Course and Room exist even if this CourseSection disappears?

Compile during class: javac *.java → java Main

Reference does not automatically mean ownership

DESIGN SEMANTICS

All three lines look similar in Java:

FIELDS

```
private Student student;
private Room room;
private MeetingSchedule
schedule;
```

But the domain meaning can differ:

Association

Student participates in the section but remains independent.

Aggregation / shared part

Room may be used by many schedules and exists independently.

Composition / owned part

MeetingSchedule can be created specifically for and owned by one CourseSection.

LECTURE 10

Association → aggregation → composition

RELATIONSHIP STRENGTH

ASSOCIATION

knows / uses

Independent objects collaborate.

AGGREGATION

shared whole–part

A whole groups parts, but parts may outlive it or be shared.

COMPOSITION

strong ownership

Whole strongly controls the part; lifetime is usually dependent.

UML relationships encode different ownership and whole–part assumptions.

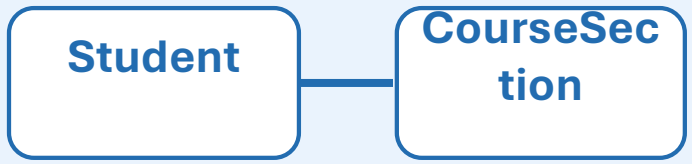
Java has no special field syntax that enforces this continuum. It is a modeling decision expressed through construction, sharing, mutation, and lifetime.

UML notation — diamond goes at the whole

DIAGRAM RULE

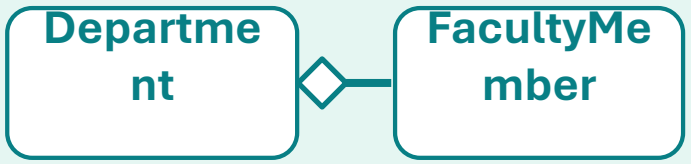
Three distinct line endings express different modeling assumptions.

ASSOCIATION



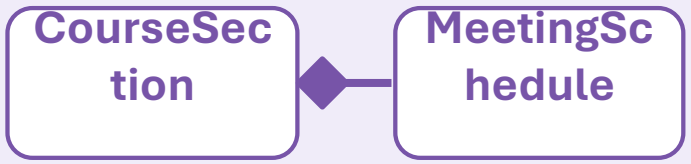
Plain line: objects participate or collaborate.

AGGREGATION



Hollow diamond: a whole groups independent parts.

COMPOSITION



Filled diamond: exclusive, strong whole–part ownership.

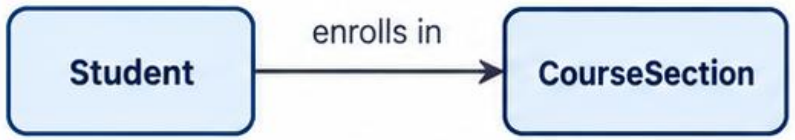
The diamond is placed beside the WHOLE, never beside the PART.

Association vs Aggregation vs Composition

Three ways to model relationships between classes (whole–part) in object–oriented design.

Association

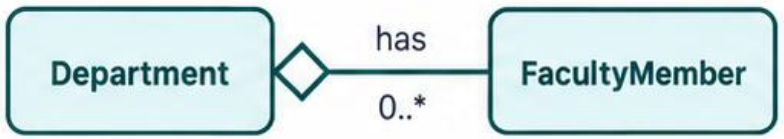
A general relationship between classes



Definition	A general relationship where one class uses or is related to another.
Ownership	No ownership.
Lifetime	Classes have independent lifetimes.
Example	A `Student` enrolls in a `CourseSection`.

Aggregation

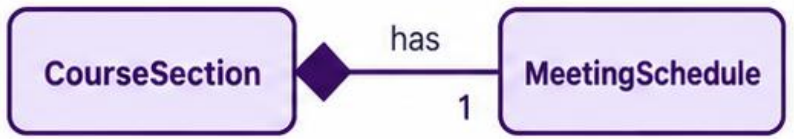
A whole–part relationship with weak ownership



Definition	A whole–part relationship where the part can exist independently.
Ownership	Weak ownership (“has-a”).
Lifetime	Parts can exist without the whole.
Example	A `Department` has `FacultyMember` (s).

Composition

A strong whole–part relationship with exclusive ownership



Definition	A strong whole–part relationship where the part cannot exist independently.
Ownership	Strong ownership (“owns”).
Lifetime	Part’s lifetime is bound to the whole (created/destroyed together).
Example	A `CourseSection` has a `MeetingSchedule`.

How to decide

Ask these questions about the part in relation to the whole:

1. **Can part exist independently?**
   If yes, it’s not composition.
2. **Can part be shared?**
   If yes, it’s aggregaition (not composition).
3. **Does the whole strongly own it?**
   If yes, it’s composition.

Association — objects know or use each other

ASSOCIATION

Example

A Student is enrolled in a CourseSection.



Lifetime

Deleting one section should not destroy the Student object.

Ownership

Neither object owns the other’s existence.

Implementation

Could be represented by a reference, ID, collection, repository lookup, or other mechanism.

Aggregation — a whole groups independently living parts

AGGREGATION

Example

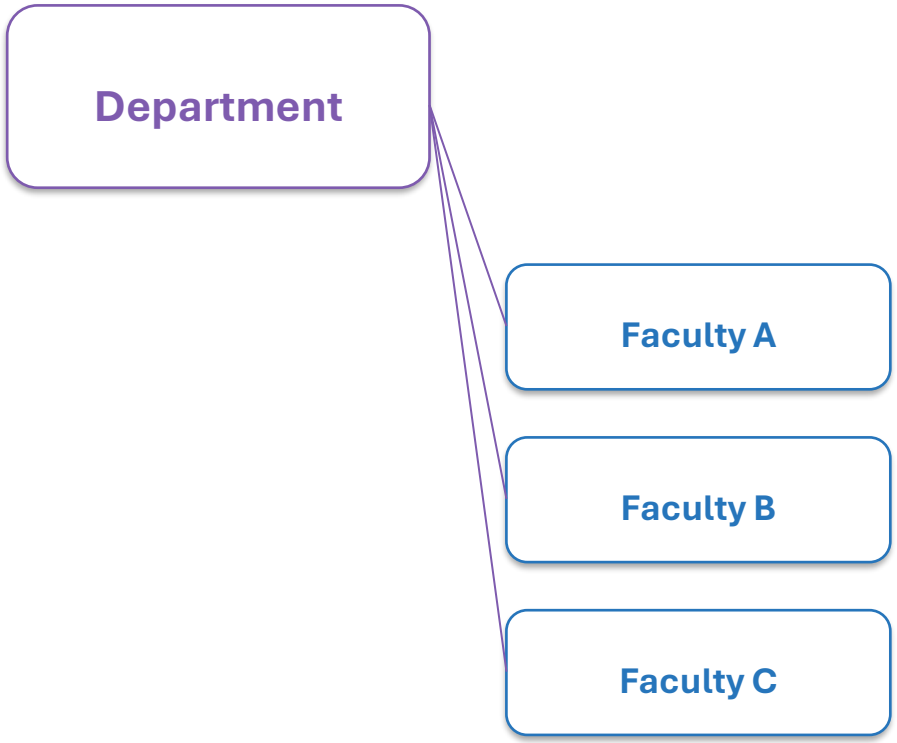
A Department groups Faculty members.

Independent lifetime

A faculty member can exist before joining the department and after leaving it.

Potential sharing / reassignment

Parts may move between wholes or be referenced elsewhere.



Modeling note: “aggregation” is often used selectively in modern design discussions. The important idea is shared/independent lifetime.

COMPOSITION

Composition — strong ownership and lifetime dependency

Example

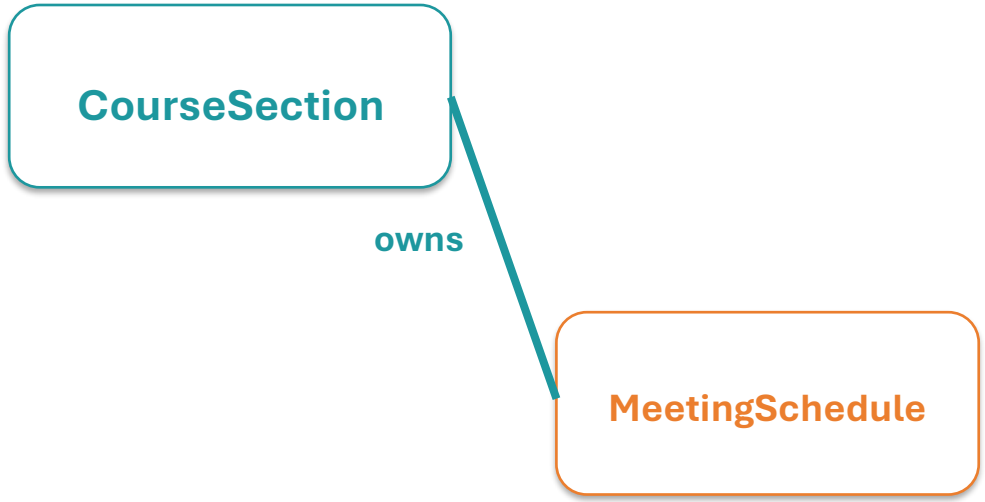
A CourseSection owns its section-specific MeetingSchedule.

Creation

The whole may create or exclusively receive the part.

Lifetime

If the CourseSection is removed from the model, its dedicated MeetingSchedule normally has no independent purpose.



Strong composition should reflect the domain — not merely the fact that one field stores another object reference.

Think–pair–share: classify the relationships

PREDICT

Classify each first; then compare your reasoning with the next slide.

Student ↔ CourseSection

Can the student outlive the section?

Department → FacultyMember

Can faculty members change departments?

CourseSection → MeetingSchedule

Is the schedule section-specific?

Order → OrderLine

Does a line item exist outside its order?

Choose: Association / Aggregation / Composition. Justify ownership and lifetime.

LECTURE 10

Quick classification — what relationship is this?

CLASS ACTIVITY

Student ↔ CourseSection — association

student exists independently

Department → Faculty — aggregation / association

faculty can move or outlive department

CourseSection → MeetingSchedule — composition

schedule is dedicated to that section

Order → OrderLine — composition

line exists as part of its order

Car → Driver — association

driver is independent

House → Room — often composition

room is structurally part of house in the model

Ask students to justify the lifetime and ownership assumptions. More than one model can be defensible if the domain assumptions differ.

Ownership and lifetime dependency

Ask three questions:

1. **Who creates the part?**

   Creation can signal ownership, but injection can still represent ownership.

2. **Can the part be shared?**

   Shared parts usually weaken the case for strong composition.

3. **Does the part have meaning after the whole disappears?**

   If no, lifetime dependency is strong.

Java reality

Garbage collection reclaims unreachable objects; it does not know your UML concept of composition. You express ownership through your object graph and API design.

Design consequence

Avoid exposing internal owned parts in ways that let unrelated clients replace or mutate them freely.

Internal creation vs constructor injection

OWNERSHIP SIGNALS

CREATED INTERNALLY

```
public CourseSection(Course course, Room room) {
   this.course = course;
   this.schedule = new MeetingSchedule(
      room, new TimeSlot("Mon", "10:00")
   );
}
```

INJECTED

```
public CourseSection(Course course,
                     MeetingSchedule
schedule) {
    this.course = course;
    this.schedule = schedule;
}
```

Internal creation

Makes exclusive ownership more obvious, but reduces flexibility.

Injection

Supports reuse/testing and can still be used with owned parts; semantics must be documented and protected.

Constructor injection does not automatically mean aggregation. Ownership is a semantic contract.

LECTURE 10

DESIGN QUALITY

Cohesion — how strongly do a class's responsibilities belong together?

High cohesion

A class has a focused purpose; its fields and methods support that purpose.

Low cohesion

A class collects unrelated data and operations simply because they were convenient to place together.

HIGHER COHESION

```
class CourseSection {
    boolean hasSeat() {...}
    void enroll(Student s) {...}
    String describeSchedule() {...}
}
```

LOWER COHESION

```
class UniversitySystem {
    void enrollStudent() {...}
    void printInvoice() {...}
    void saveFile() {...}
    void sendEmail() {...}
}
```

Heuristic: a class name should make most of its responsibilities feel unsurprising.

Refactor low cohesion by separating responsibilities

REFACTOR

BEFORE — RegistrationSystem

enrollStudent()
calculateFee()
sendEmail()
saveTranscript()
allocateRoom()

split by responsibility

EnrollmentService

enroll()

FeeService

calculate()

Notifier

send()

TranscriptRepository

save()

RoomAllocator

allocate()

Result: each class has a clearer reason to change.

Coupling — how much does one class depend on another?

DESIGN QUALITY

Low coupling

A class knows only what it needs and depends on stable, small public behavior.

High coupling

A class depends on many details, concrete representations, or internal state of other objects.

LOWER COUPLING

```
class ReportPrinter {
    void print(CourseSection section) {

System.out.println(section.describe());
    }
}
```

HIGHER COUPLING

```
class ReportPrinter {
    // reaches into many fields / nested
details
    // duplicates formatting rules
    // assumes internal representations
}
```

Goal: enough collaboration to do useful work — not zero coupling.

Nested object graphs can accidentally increase coupling

COUPLING RISK

LONG NAVIGATION CHAIN

```
String roomCode = section
    .getSchedule()
    .getRoom()
    .getBuilding()
    .getCode();
```

Why risky?

Client code knows the internal path through several objects. A structural change can break many callers.

Better direction

Ask the object that already knows the collaboration: section.getMeetingLocation() or section.describe().

Important nuance

Do not blindly forbid chaining. Use it as a signal to ask whether responsibility is leaking across boundaries.

Reduce coupling — ask for behavior, not internals

REFACTOR

Long reference chains reveal knowledge of internal object structure.

TIGHTLY COUPLED

Client navigates several implementation details.

```
section.getSchedule()
  .getRoom().getCode();
```

LOWER COUPLING

Section exposes a useful operation instead.

```
section.getMeetingLocation();
```

Responsibility stays inside CourseSection; callers depend on its public API, not the entire object graph.

LECTURE 10

Good class design: high cohesion + manageable coupling

DESIGN TARGET

COHESION **COUPLING**

Focused class

Responsibilities belong together.

Clear collaboration

Public behavior hides unnecessary details.

Stable boundaries

Changes stay local whenever possible.

A practical review checklist:

Can I explain the class purpose in one sentence?

Do most methods use/support the same core state?

Does the class know fewer collaborators than necessary?

Would changing one collaborator force edits everywhere?

Can I hide internal nested objects behind meaningful behavior?

LECTURE 10

Favor composition when inheritance is not a true IS-A

Bad question: "Can I reuse code by extending this class?"

Better question: "Is the new type truly substitutable as an instance of the parent concept?"

IS-A

Car is a Vehicle.

HAS-A

Car has an Engine.

WRONG INHERITANCE

Car extends Engine — code reuse does not make the domain relationship valid.

Composition often changes behavior by replacing a collaborator instead of changing the inheritance hierarchy.

LECTURE 10

Design decision clinic — inheritance or composition?

CLASS ACTIVITY

Report ↔ PDFExporter — HAS-A / uses

Ask: “is-a” or “has-a / uses-a”?

ElectricCar ↔ Vehicle — IS-A

Ask: “is-a” or “has-a / uses-a”?

CourseSection ↔ Room — HAS-A / association

Ask: “is-a” or “has-a / uses-a”?

SmartPhone ↔ CameraModule — HAS-A / composition

Ask: “is-a” or “has-a / uses-a”?

TeachingAssistant ↔ Student — depends on domain; often role relationship

Ask: “is-a” or “has-a / uses-a”?

Stack ↔ Array — HAS-A implementation detail

Ask: “is-a” or “has-a / uses-a”?

The goal is reasoning, not memorizing a table.

Why cloning appears in a composition lecture

Simple object

Copying primitive/string-like state is straightforward.

Composed object

The object contains references to other mutable objects.

Copy question

Should the new object share those nested objects or receive independent copies?

This is exactly where “shallow” vs. “deep” copy matters.

REFERENCE ASSIGNMENT

```
CourseSection copy = original;   // NOT a clone
// both variables refer to the same CourseSection object
```

LECTURE 10

CLONING

Java cloning model — Object.clone() + Cloneable

Object.clone()

Object defines a protected clone() method that performs a field-by-field copy.

Cloneable

A marker interface used to indicate that Object.clone() is permitted for instances of the class.

Important consequence

Reference-valued fields are copied as references. The nested objects are not automatically cloned.

Exception behavior

If a class does not implement Cloneable and Object.clone() is invoked, CloneNotSupportedException is thrown.

Marker-interface mechanics are previewed here because the CDF includes Cloneable; interfaces are studied formally later.

Minimal cloneable class — expose clone carefully

CODE

TimeSlot.java

```
public class TimeSlot implements Cloneable {
    private String day;
    private String startTime;

    @Override
    public TimeSlot clone() {
        try {
            return (TimeSlot) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }
}
```

Why public override?

Object.clone() is protected, so client code cannot normally call it directly. The subclass can provide a public wrapper/override.

Why cast?

super.clone() returns Object, so a covariant TimeSlot return type is exposed after the cast.

Constructor note

This illustrates the Cloneable API. A mutable child object better demonstrates why shallow copies can be risky.

Three operations — three reference graphs

Watch the object identities before predicting mutations.

1 Reference assignment

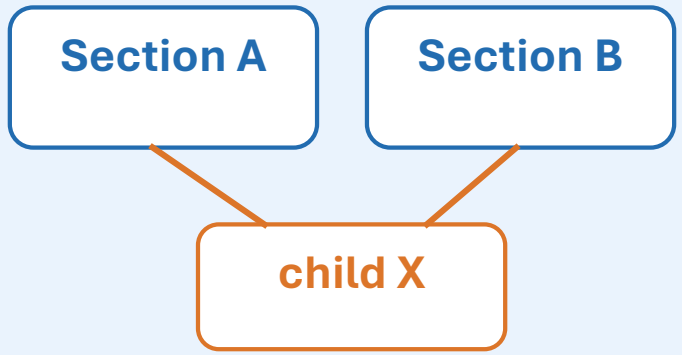
```
b = a
```



ONE outer object

2 Shallow clone

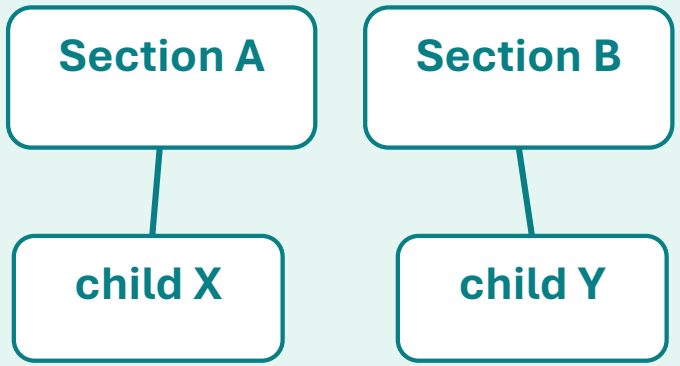
```
b = a.clone()
```



TWO outer objects; child shared

3 Independent nested copy

```
b = new Section(a)
```



TWO outer objects; child copied

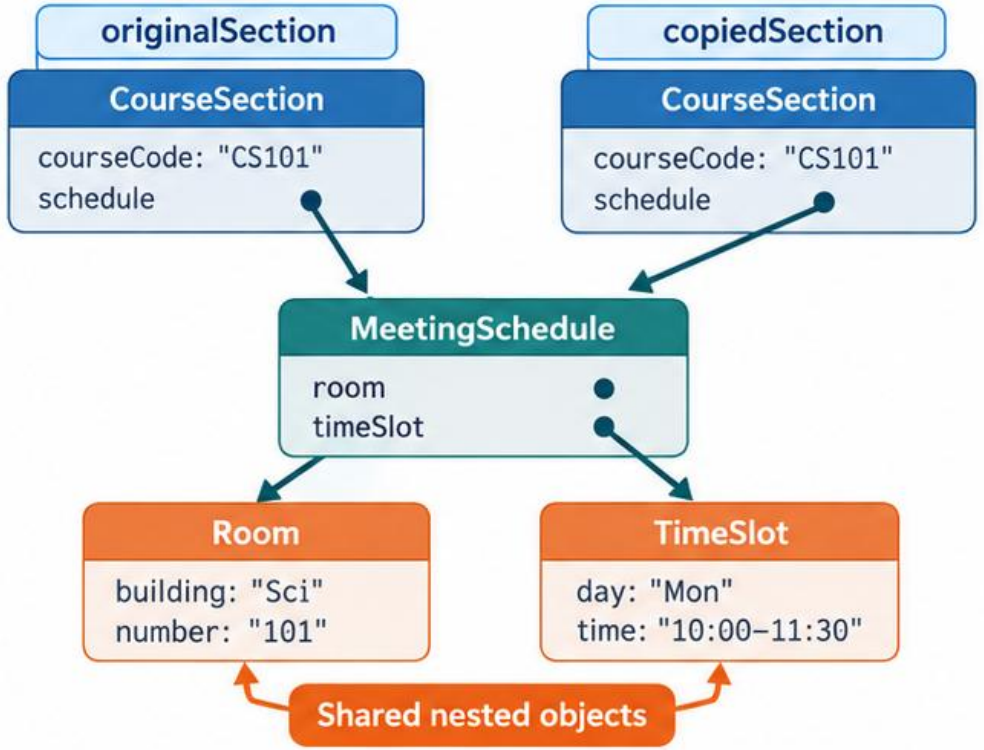
A deep copy is a policy, not a Java operator. Deliberately choose which nested objects to copy or share.

Shallow Copy vs Deep Copy in Composed Objects

When an object contains other objects, how do copies handle those nested objects?

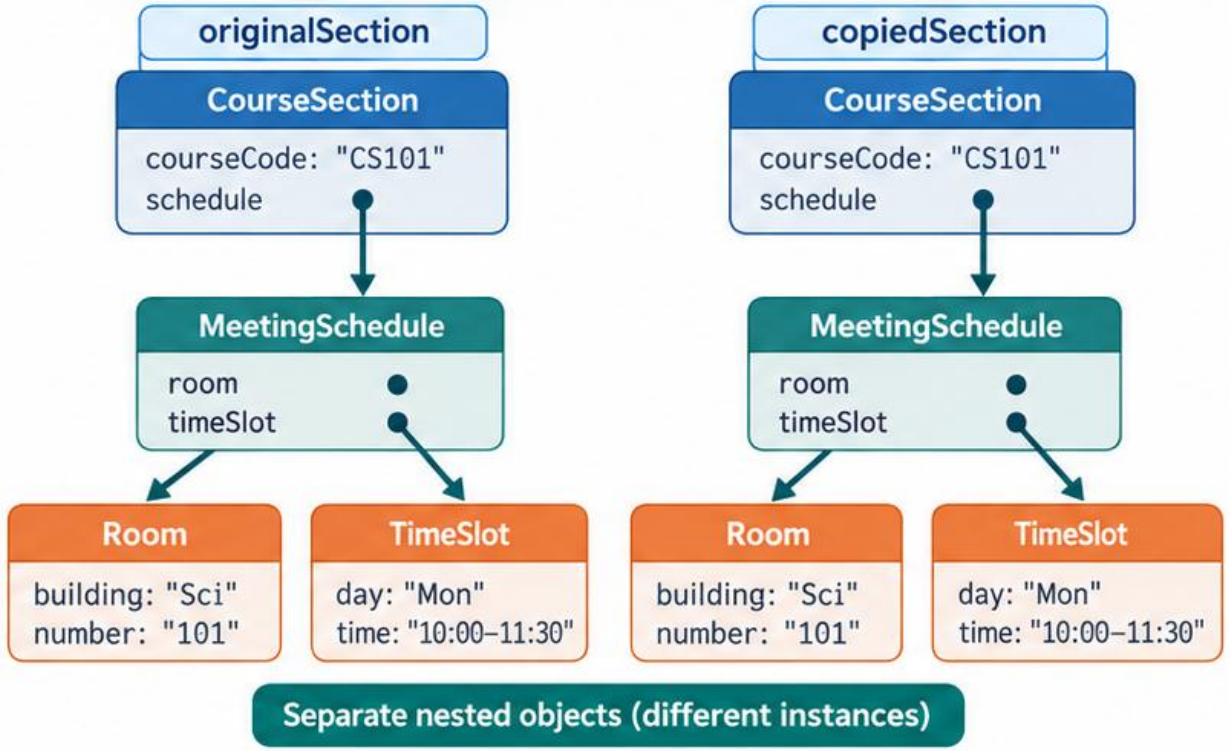
Shallow Copy

outer objecty copied, nested references shared



Deep Copy

outer objecty copied, nested objects also copied



What happens if one nested object is modified?

1. In a shallow copy, the change is visible in both the original and the copy (shared object).
2. In a deep copy, the change affects only the modified object (the copies are independent).
3. Shallow copy is faster and uses less memory, but can lead to unintended side effects. Deep copy is safer when independent copies are needed.

Key idea **Assignment copies references only; copying composed objects requires an explicit copying strategy.**

Live coding — observe a shallow clone

LIVE CODING

ShallowCloneDemo.java

```
// Standalone simplified MeetingSchedule for cloning:
class MeetingSchedule implements Cloneable {
   Room room;
   TimeSlot slot;
   MeetingSchedule(Room r, TimeSlot t) {
      room = r; slot = t;
   }
   @Override public MeetingSchedule clone() {
      try { return (MeetingSchedule) super.clone(); }
      catch (CloneNotSupportedException ex) {
         throw new AssertionError(ex);
      }
   }
}
// In main:
MeetingSchedule a = new MeetingSchedule(room, slot);
MeetingSchedule b = a.clone();
System.out.println(a == b);         // false
System.out.println(a.room == b.room); // true
```

PREDICT 1

a == b → false

PREDICT 2

a.room == b.room → true

Meaning

Outer object identity differs; the nested Room object is shared.

Change the shared Room and observe both schedules.

DEEP COPY

Deeper copies — choose a strategy explicitly

Clone nested object

copy.room = room.clone();
requires nested type to support safe copy

Copy constructor

new Room(other.room)
often clearer and constructor-driven

Factory method

Room.copyOf(other.room)
names the copying intent

Immutable nested object

share safely
best when nested state cannot change

There is no universal “deep clone” operator in Java. You decide which nested parts should be copied, shared, or treated as immutable.

Copy constructor — copy owned state, share independent state

CODE

```
COPY CONSTRUCTOR

public MeetingSchedule(MeetingSchedule other) {
   this.room = other.room;            // share Room
   this.slot = new TimeSlot(other.slot); // copy slot
}

MeetingSchedule copy = new
MeetingSchedule(original);
// copy != original; copy.getRoom() ==
```

Advantages

- explicit construction path
- easy to choose which nested objects are copied
- no CloneNotSupportedException
- fits normal constructor reasoning

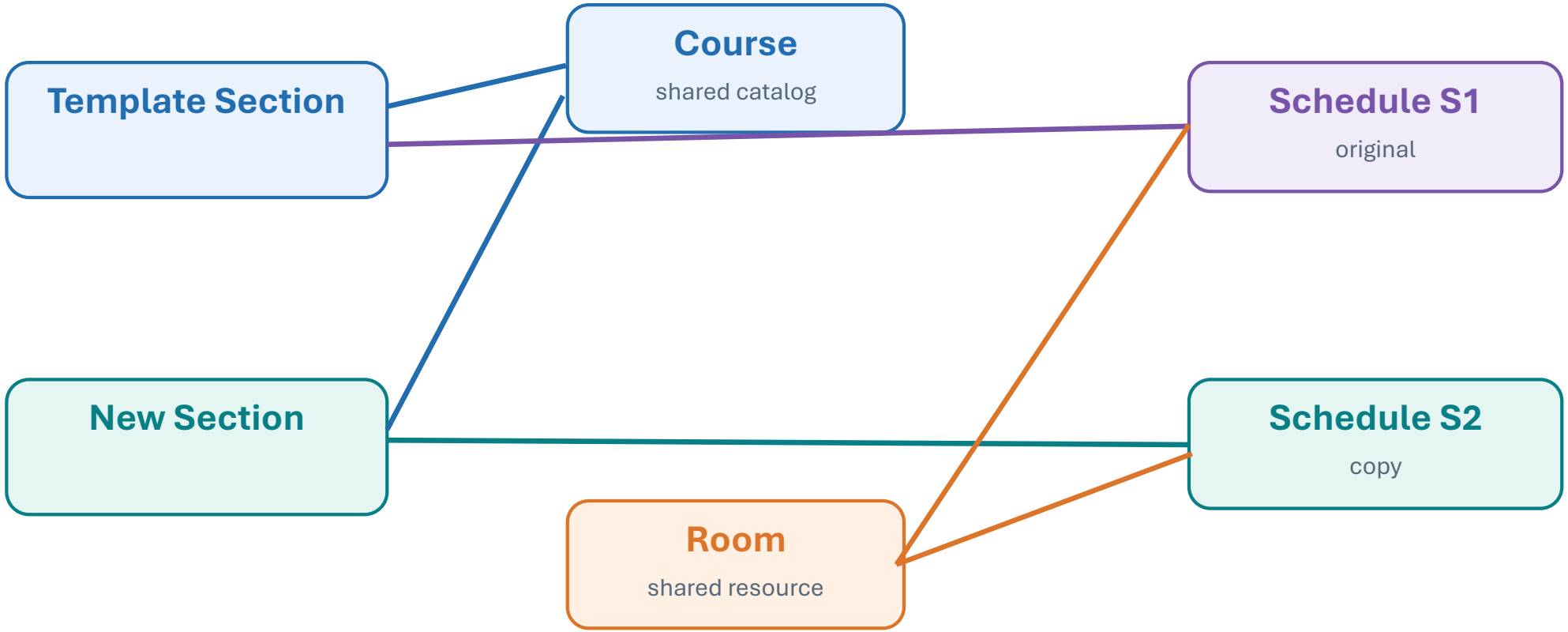
Trade-off

Copy policies follow ownership: share the independent Room; copy the dedicated TimeSlot. Java GC does not enforce UML ownership. Because TimeSlot is immutable here, sharing it would also be safe.

Realistic copy policy — copy what the section owns

COPY STRATEGY

An independent copy does not have to duplicate every object in the world.



Course and Room can stay shared; each section gets a separate schedule (and its dedicated mutable parts).

Cloning misconceptions

MISCONCEPTIONS

Assignment copies an object

No — it copies only the reference.

clone() is automatically deep

No — Object.clone() is shallow for reference fields.

Cloneable defines clone()

No — it is a marker interface; clone() comes from Object.

Constructors run normally during clone

Not in the ordinary object-construction path.

Every class should support cloning

No — expose copying only when it makes semantic sense.

Deep copy means copy absolutely everything

Not necessarily — immutable/shared collaborators can remain shared by design.

LECTURE 10

DESIGN + CODE

Integrated design challenge — clone a section template

Scenario: The department creates a section template containing a Course reference and a section-specific MeetingSchedule. A new section is created from the template.

Share

Course — same catalog course object can be referenced by multiple sections.

Copy

MeetingSchedule — each section should get its own schedule object.

Decide

Room — may be shared; TimeSlot — copy for independent section scheduling.

DESIGN TARGET

```
CourseSection copy = new CourseSection(template);
// share Course
// copy MeetingSchedule
// preserve independent sectionId
```

Ask students: Which fields should be shared? Which should be independently copied? Why?

Lecture 10 live-coding workflow

LIVE CODING

Repository location

00_Course_Master\01_Lectures\Week05\L10\LiveCoding\01_NestedComposition

TERMINAL

```
javac *.java
java Main

// then modify cloning demo
javac *.java
java ShallowCloneDemo
```

1 Build graph

Course → Section → Schedule → Slot/Room

2 Trace identity

Use == to compare outer and nested objects

3 Mutate

Change a shared nested object

4 Refactor copy

Create an independent nested copy

Predict → code → compile → run → explain the object graph.

5-minute exit challenge

CHECK UNDERSTANDING

Ask students to sketch the references before stating the output.

```
Course course = new Course("CSC241", "OOP");
Room room = new Room("A-201");
TimeSlot time = new TimeSlot("Mon", "10:00");
MeetingSchedule a = new MeetingSchedule(room, time);
MeetingSchedule b = a.clone();

System.out.println(a == b);
System.out.println(a.getRoom() == b.getRoom());
```

PREDICT & EXPLAIN

1. How many Schedule objects?

2. How many Room objects?

3. Why are the two Boolean outputs different?

4. What changes after a selective copy?

LECTURE 10

RECAP

Recap + exit ticket

Remember

Multi-level composition creates nested object graphs.

Association, aggregation, and composition differ mainly in semantics, ownership, and lifetime.

High cohesion keeps responsibilities focused.

Manage coupling by hiding unnecessary internal structure.

Prefer composition when inheritance does not express a true IS-A.

Object.clone() performs a shallow field copy; nested references remain shared unless copied explicitly.

Exit ticket

1. Give one example of association and one of composition.
2. Why can two Java fields look identical but model different relationships?
3. What is the difference between high cohesion and low coupling?
4. After shallow clone, what happens to a mutable reference field?
5. When would you prefer a copy constructor?

Next: Lecture 11 — UML Class Diagrams